Seminar 04

13.11.2026

Lernziel: WebSockets implementieren für ein einfaches Multiplayer-Spiel.

1. WebSocket Game: Setup

Der Starter ist ein lokales Spiel mit zwei Spielern, die auf einem Canvas angezeigt werden. Daraus soll ein echtes Multiplayer-Spiel werden. Der Node.js Server stellt dabei den zentralen Datenort dar. Keiner der Clients soll seine Position selbst ausrechnen.

Setup, fünf Dateien:

- `index.html`: Startseite, lädt `styles.css` und `client.js`
- `client.js`: der Client; **Schritt 1** (Starter ausführen, Game Loop verstehen) läuft sofort
- `styles.css`: fertiges Styling (**nicht verändern**)
- `server.js`: Stub mit TODOs; startet erst mit einer echten `WebSocketServer`-Instanz
- `package.json`: `"type": "module"` und die Abhängigkeit `ws`

1. Führt im Aufgabenordner `npm install` aus
2. Startet den Server mit `npm start`, er läuft auf `ws://localhost:8080`
3. Öffnet `index.html` in **zwei** Browser-Tabs, idealerweise nebeneinander

Das Nachrichtenprotokoll im Überblick:

Nachricht	Richtung	Inhalt
`welcome`	Server ⟶ neuer Spieler	eigene ID, Startposition und Farbe
`state`	Server ⟶ alle	komplette Liste aller Spieler, bei jeder Änderung
`move`	Spieler ⟶ Server	gewünschter Schritt: `dx`, `dy`
`lobby-full`	Server ⟶ dritter Spieler	Lobby ist voll, Verbindung wird geschlossen
`collision`	Server ⟶ beide Spieler	die zwei Spieler überlappen

2. Client: Verbindung und Bewegung

2.1. Verbindung öffnen (TODO 1)

Bekannt aus der Vorlesung: ein WebSocket verbindet genau einmal und bleibt offen.

```javascript
const socket = new WebSocket("ws://localhost:8080");
```

Aktualisiert `#status` bei `open`, `error` und `close`: zum Beispiel **nicht verbunden**, **verbunden** und **getrennt**.

2.2. State empfangen (TODO 2)

- Bei `welcome` merkt ihr euch eure `id`: nur so erkennt ihr später auf dem Canvas, welcher Spieler **ihr** seid.
- Bei `state` ersetzt ihr `players` komplett durch die empfangene Liste. Der Game Loop zeichnet dann einfach weiter, nur dass die Positionen jetzt vom Server kommen.

```
1   socket.onmessage = (event) => {                                    JavaScript
2       const msg = JSON.parse(event.data);
3       // TODO: welcome → id speichern, state → players ersetzen
4   };
```

2.3. Bewegung senden (TODO 3)

- Entfernt die lokale Steuerung von Player 2. Jeder Spieler sendet nur noch seine **eigenen** Bewegungen.
- Sendet **genau eine** Nachricht pro Tastendruck (`keydown`), **nicht** pro Frame.

```
1   socket.send(JSON.stringify({ type: "move", dx, dy }));             JavaScript
```

3. Server: Spieler und State

3.1. Server instanziieren

Ersetzt das `null` durch eine echte Instanz. Erst dann startet der Server und das `connection`-Event feuert.

```
1   const wss = new WebSocketServer({ port: PORT });                  JavaScript
```

3.2. Spieler akzeptieren (TODO 2)

- Es sind nur 2 Spieler erlaubt: ab dem dritten bekommt der neue Client `lobby-full`, dann schließt ihr die Verbindung.
- Vergibt ID, Farbe und Startposition aus den vorhandenen Konstanten und speichert den Spieler in der Map.
- `welcome` geht nur an den **neuen** Spieler. Danach einmal `state` an alle broadcasten.

3.3. Bewegung verarbeiten (TODO 3)

- Aktualisiert die Position des **Senders** und clamped sie an die Canvas-Grenzen — **inklusive Spielergröße**: 0..620 statt 0..640.
- Broadcastet danach den Zustand ALLER Spieler an ALLE Clients. Achtung: Der `state` muss nach jeder Änderung auf dem neuesten Stand sein, sonst sehen die Clients Ghosts.
- **Bonusfrage**: Warum sollte der Server `dx` und `dy` aus der Nachricht nicht ungeprüft übernehmen? Was könnte ein manipulierter Client sonst erreichen?

3.4. Verbindungsende behandeln (TODO 5)

Geht ein Spieler (Tab zu, Browser zu), entfernt ihn aus der Map und broadcastet den aktualisierten `state` — die Übrigen sollen den Abgang sofort sehen.

4. Server: Kollision (optional)

4.1. TODO 4: Treffer

Wenn die beiden Spieler (je 20 x 20 Pixel) überlappen, sendet `collision` an beide. Tipp: Überlappung heißt **in x und in y** gleichzeitig.

```
1   Math.abs(a.x - b.x) < SIZE && Math.abs(a.y - b.y) < SIZE           JavaScript
```

5. Debugging und Ressourcen

Fehlersuche:

- In den Browser-DevTools (F12) gibt es unter **Network** einen WS-Filter: dort seht ihr jeden Frame in beide Richtungen live mit.

- Server-Änderungen brauchen einen Neustart (`Ctrl+C`, dann erneut `npm start`) — der Client reicht einen Browser-Reload.
- Steht im `#status` dauerhaft **nicht verbunden**: läuft der Server wirklich? Wurde `npm install` ausgeführt?

Zum Nachschlagen:

- MDN: WebSocket-Client: Events, `send()`, `readyState`
- ws-README: serverseitige Beispiele, fast 1:1 für TODO 2 und 3 nutzbar
- javascript.info: kompakte Einführung
- MDN: Tastenwerte: warum Pfeiltasten `"arrowup"` heißen